

A Governed Deformable Radial Membrane for Compute-Aware AI Personalization

Condensed publication version of the Dynamic Radial Membrane v2.6 research artifact

Don Michael Feeney Jr.

June 18, 2026

Abstract

This paper presents a governed deformable radial membrane architecture for compute-aware AI personalization. The model represents adaptive behavior as a circular dynamical surface composed of phase-positioned behavioral strings rather than as a static personality preset, preference vector, or prompt-level style layer. Each string contributes to expression, reasoning depth, context use, tool likelihood, and compute cost. A governor routes activation through coherent V-shaped channels, applies inverse-cost pressure, bounds reasoning depth, suppresses unnecessary expansion, and tracks a Lyapunov-style stability signal. The architecture further extends the membrane from a fixed circular boundary to a deformable bidirectional polar surface: sustained behavioral load may stretch the boundary outward, while cost pressure or governor suppression may collapse it inward. Tangent expressivity, curvature, asymmetry, and radius deviation therefore become measurable control signals. Cost is modeled as a multidimensional runtime vector, enabling reduction of avoidable compute while preserving useful reasoning, safety, accessibility, correctness, and user-requested depth. A lightweight toy simulation, Project Rainbow, demonstrates that the proposed state variables, routing rules, governor dynamics, and ablation conditions can be parameterized and inspected under simplified workloads. The contribution is a conceptual and mathematical framework for aligning personalization, runtime control, and compute efficiency, with a staged validation roadmap for future prototype evaluation.

1. Introduction

AI personalization is often treated as a surface adjustment: tone, remembered facts, response length, or a static personality profile. Compute management is typically treated separately through token budgets, retrieval limits, tool-call thresholds, latency controls, and infrastructure cost. In practice, these concerns are coupled. A preference for concise answers changes token expenditure; a need for technical depth changes reasoning depth, context use, and verification cost. Personalization that is not governed can therefore expand compute in opaque and unstable ways.

This paper proposes a unified representation for these dynamics: a governed deformable radial membrane for compute-aware AI personalization. The model treats AI behavior as a geometric control surface composed of phase-positioned behavioral strings. Each string represents a

controllable behavioral dimension—such as depth, precision, context sensitivity, initiative, exploration, creativity, tone, transparency, technical detail, structural rigor, emotional warmth, or conciseness—and carries a corresponding compute signature.

The radial membrane provides a compact state representation. Behavioral strings are arranged around a circular phase domain, and activation propagates through coherent V-shaped channels between related strings. This enables aligned behavioral modes rather than discontinuous switching among isolated traits. Technical work may route through depth, precision, technical detail, and structural rigor; interpersonal work may route through tone, emotional warmth, and conciseness.

A governor layer stabilizes the membrane. It evaluates local cost, phase alignment, reasoning depth, curvature, tension, stiffness, and task value before allowing activation to expand. It applies inverse-cost routing, bounded-depth updates, optional-expansion suppression, and Lyapunov-style energy tracking. The objective is not blind compute minimization, but reduction of avoidable compute while preserving task success, safety, accessibility, correctness, user trust, and requested depth.

The principal architectural extension is a deformable bidirectional membrane boundary. Rather than treating the outer boundary as a fixed compute envelope, the model treats it as an active geometric signal. Sustained behavioral load can stretch the boundary outward, while cost pressure, suppression, or reduced coherence can collapse it inward. Radius deviation, tangent direction, curvature, asymmetry, and expressivity therefore become measurable indicators of the system’s operating geometry.

This geometry is paired with a multidimensional cost taxonomy. Cost is modeled as a vector of observable runtime signals, including generated tokens, reasoning depth, context-window use, retrievals, tool calls, latency, correction overhead, and instability-recovery cost. The governor projects these signals into weighted observable cost while preserving a quality floor. Additional compute is justified when it improves correctness, safety, coherence, or user value; it is suppressed when it reflects verbosity, redundant context, incoherent routing, repeated correction, or low-value expansion.

The paper contributes a conceptual and mathematical architecture for aligning personalization, runtime control, and compute efficiency. A lightweight toy simulation, Project Rainbow, parameterizes the twelve-string membrane, task signatures, curvature-weighted routing, governor updates, ablations, and stability traces under simplified workloads. These results are not production validation; they demonstrate that the proposed variables and controls can be expressed, simulated, inspected, and challenged before prototype deployment.

The remainder of the paper is organized as follows. Section 2 motivates the architecture and positions it against static personalization and isolated compute controls. Section 3 defines the radial identity membrane and behavioral string model. Section 4 introduces V-channel routing

and governed flow. Section 5 formalizes the governor and stability controls. Section 6 develops the deformable and bidirectional boundary geometry. Section 7 presents the cost taxonomy and quality-preserving reduction layer. Section 8 concludes by summarizing the contribution, noting the Project Rainbow toy simulation, and outlining the validation path and limitations.

2. Background and Motivation

Modern AI assistants operate across tasks, applications, documents, conversations, tools, and user-specific contexts. The design challenge is not only whether a model can produce a correct answer, but how its behavior should be shaped at runtime: how much context to use, how deeply to reason, when to retrieve, when to act, how much to explain, and how to adapt without becoming unstable, opaque, or unnecessarily expensive. Personalization, inference control, safety, and compute cost therefore interact during each request.

Existing personalization methods often add user adaptation as a preference layer: tone settings, response-length preferences, remembered facts, instruction profiles, or static personality presets. These mechanisms improve experience but usually do not explain how personalization changes computational load. Technical detail may increase retrieval, verification, and output length; concision may reduce tokens but also suppress useful explanation; proactive assistance may increase planning and tool-use likelihood. Behavioral adaptation is therefore also compute adaptation.

Conversely, many compute-control mechanisms improve efficiency without modeling user-shaped behavior in the same state space. Token limits, retrieval budgets, latency controls, response-length constraints, routing rules, and adaptive inference policies may reduce resource use, but they often act as external constraints. Compute reduction is valuable only when it preserves task success, correctness, safety, accessibility, trust, and required depth.

This motivates a shared representation in which behavioral preference and compute pressure are evaluated together. The radial identity membrane treats AI behavior as a governed geometric state rather than as a fixed personality preset or set of independent controls. Behavioral strings define controllable dimensions of expression and reasoning; V-shaped channels define coherent routes; the governor applies cost, policy, stability, and task-value constraints; and the deformable boundary exposes curvature, asymmetry, tangent direction, outward stretch, and inward collapse.

The geometry is not merely illustrative. It represents continuity, coupling, and constraint. Related behaviors can blend through coherent corridors; high-cost behaviors remain available when justified but become harder to sustain without task value; repeated coherent patterns can influence future readiness without converting every temporary interaction into durable personalization. Activation indicates behavior, radius indicates reasoning reach, thickness indicates compute load, curvature indicates transition pressure, and boundary deformation indicates how the system is shaped under load.

The model also draws from control systems. An adaptive AI system should expand capability when justified and recover when task pressure subsides. The governor functions as a dissipative control layer: it permits task-driven expansion, then biases the system toward lower energy, coherent routing, bounded compute, and policy-valid behavior. This is especially relevant at enterprise scale, where small inefficiencies in reasoning depth, context expansion, retrieval, or tool use can accumulate across high interaction volume.

The deformable boundary extends this control view. A fixed circle cannot represent asymmetric behavioral load. In realistic interaction, some regions are stretched by sustained demand while others contract under cost pressure, suppression, or reduced coherence. By allowing outward and inward deformation, the architecture makes boundary shape a governor-readable signal. Identity becomes a geometry of behavioral pressure, recovery, and control.

The resulting framework addresses a central problem: how to make adaptive AI behavior more personal without making it less governable, more capable without making it less efficient, and more expressive without making it less measurable. The model is not presented as a production implementation or deployment-scale empirical result. It is a conceptual and mathematical framework, supported by a toy simulation, for examining whether personalization, runtime control, and compute efficiency can be unified through a shared geometric governor.

3. Radial Identity Membrane Model

The radial identity membrane is the central representational object of the architecture. It models adaptive AI behavior as a circular dynamical surface rather than as a fixed personality preset, isolated preference vector, or collection of independent style controls. The membrane provides a shared state space in which behavioral expression, reasoning depth, context use, tool likelihood, compute cost, and governor control can be represented together.

At the architectural level, the membrane is composed of twelve phase-positioned behavioral strings arranged around a circular domain. Each string corresponds to a controllable dimension of behavior, such as depth, precision, context sensitivity, initiative, exploration, creativity, tone, transparency, technical detail, structural rigor, emotional warmth, or conciseness. These strings are not intended to define human personality traits. They define runtime behavioral modes that affect how the AI responds, how much compute it spends, and how strongly it should be governed under a given task.

Let the circular phase domain be indexed by angular position $\theta \in [0, 2\pi)$. Each behavioral string s_i occupies a fixed phase position:

$$\theta_i = \frac{2\pi i}{12}, i = 1, \dots, 12$$

The instantaneous identity state can be represented by a coefficient vector:

$$\mathbf{a}(t) = [a_1(t), a_2(t), \dots, a_{12}(t)]$$

where $a_i(t)$ is the activation coefficient of behavioral string s_i at time t . A higher coefficient indicates stronger expression of that behavioral mode, but the coefficient alone does not determine output. Each string also carries geometric and compute-relevant quantities such as reasoning radius, local cost, tension, stiffness, and damping. The identity state is therefore not a flat preference profile; it is a governed dynamical state.

The membrane activation over the circular domain can be approximated as a superposition of behavioral basis functions:

$$u(\theta, t) = \sum_{i=1}^{12} a_i(t) \phi_i(\theta)$$

where ϕ_i is the angular basis function associated with string s_i . This equation gives the architecture its first important property: behavior is blended rather than discretely switched. A response can express depth and precision together, or warmth and conciseness together, because multiple strings may activate concurrently and nearby strings may share resonance through the membrane.

The twelve strings can be summarized as follows:

Behavioral String	Function	Compute Signature
Depth	Multi-step reasoning, inference chains, abstraction	High reasoning-depth cost
Precision	Correctness, qualification, verification	Verification and claim-checking cost
Context Sensitivity	Prior conversation, memory, documents, situational cues	Context-window and retrieval cost
Initiative	Proactive planning, next steps, tool/action suggestions	Tool and agentic-planning cost
Exploration	Branching ideation and alternative paths	Branching and token cost
Creativity	Novel synthesis, metaphor, generative variation	Generative variation cost
Tone	Formality, warmth, confidence, interpersonal style	Low expressive modulation cost
Transparency	Visible structure, explanation, rationale	Output and explanation cost

Behavioral String	Function	Compute Signature
Technical Detail	Domain terms, formulas, dense structured explanation	Detail, retrieval, and formatting cost
Structural Rigor	Organization, hierarchy, consistency	Planning and stabilization cost
Emotional Warmth	Encouragement, empathy, affective resonance	Low expressive cost
Conciseness	Compression, brevity, minimalism	Cost-reducing contraction

This table is intentionally compact for the publication version. The full artifact expands each string’s compute signature and geometric role, but the condensed paper only requires the reader to understand that each string is both behavioral and compute-bearing. A request for expanded technical analysis activates a different region of the membrane than a request for a concise supportive response, and those regions carry different cost implications.

The membrane also distinguishes three geometric quantities that are often conflated in ordinary personalization language. Amplitude represents the intensity of a behavioral mode. Radius represents reasoning depth or cognitive reach: how far the system extends into inference, context, retrieval, or planning. Thickness represents compute load: the token, context, tool, latency, or stability cost required to sustain that behavior. This separation is important because a behavior can be expressive without being deeply expensive, or deep without being verbose. For example, tone may have high expressive amplitude but low reasoning radius, while technical detail may increase both radius and thickness.

At initialization, the membrane begins centered and neutral:

$$A_i(0) = 0, R_i(0) = 0, C_i(0) = C_{\text{baseline}}$$

where A_i is behavioral amplitude, R_i is reasoning radius, and C_i is local cost. This initialization means the system has behavioral structure but no strongly expressed identity state before user interaction or task pressure. The membrane is ready to respond, but it does not pre-load a fixed personality. Early user interactions, task types, and explicit preferences act as initial conditions that begin shaping which strings activate, which corridors become easier to enter, and which regions remain quiet.

This gives the model two useful properties. First, it is compute-efficient by default: no behavioral string consumes expanded reasoning depth, retrieval scope, token output, or tool likelihood until task pressure justifies it. Second, it is adaptable without being rigid: repeated coherent interactions can gradually bias the membrane toward certain routes, but temporary or unusual prompts need not become durable identity changes.

The membrane can also be interpreted regionally. The twelve strings may be grouped into four higher-level quadrants:

- Analytical: depth, precision, technical detail, structural rigor.
- Contextual: context sensitivity, transparency, situational awareness.
- Generative: initiative, exploration, creativity.
- Interpersonal: tone, emotional warmth, conciseness.

These quadrants do not replace the string-level model. They provide a diagnostic layer for interpreting activation patterns. For example, a task may activate the analytical quadrant broadly, while the governor determines whether the activation should concentrate in depth, precision, technical detail, structural rigor, or some coherent corridor among them.

The key architectural claim of this section is that identity should be represented as a trajectory through coefficient space rather than as a fixed setting. A governed update can be written abstractly as:

$$\mathbf{a}(t + 1) = \mathbf{a}(t) + \Delta \mathbf{a}^{gov}(t)$$

where $\Delta \mathbf{a}^{gov}(t)$ is the update that remains after task excitation, phase coupling, cost pressure, tension, stiffness, damping, policy constraints, and governor suppression have acted on the membrane. This means the AI's identity state is not simply what the user asked for and not simply what the model would naturally produce. It is the controlled result of behavior, cost, and stability being evaluated together.

In short, the radial identity membrane provides the paper's core abstraction: a shared geometric state in which user-shaped behavior and compute-aware governance can be modeled together. The next section builds on this state space by explaining how activation flows through coherent V-shaped channels rather than switching discontinuously between isolated behavioral modes.

4. V-Channel Routing and Governed Flow

The radial identity membrane does not treat behavioral strings as isolated switches. Because the strings occupy fixed phase positions on a continuous circular field, activation can move between neighboring or related strings through structured corridors. These corridors are called V-channels. A V-channel is a low-resistance reasoning path formed by overlap between aligned behavioral modes, allowing the system to blend behavior continuously rather than transition abruptly from one mode to another.

At a conceptual level, V-channels explain how a response can combine multiple behavioral dimensions while remaining coherent. A technical review may move through Depth, Precision, Technical Detail, and Structural Rigor. A supportive short reply may move through Tone, Emotional Warmth, and Conciseness. A planning task may move through Initiative, Context

Sensitivity, Structural Rigor, and Transparency. In each case, the system is not activating a single string; it is routing through a corridor of related strings whose combined activation produces the response posture.

Let s_k be an active source string and s_j be a candidate target string. In the simplest form, propagation from k to j depends on the target string's activation and its phase alignment with the source:

$$P_{k \rightarrow j} = f(A_j) \cdot g(|\phi_k - \phi_j|)$$

where $P_{k \rightarrow j}$ is the base propagation pressure, A_j is the activation of the target string, ϕ_k and ϕ_j are their phase positions, $f(A_j)$ increases with target activation, and $g(|\phi_k - \phi_j|)$ decreases as phase distance grows. Nearby or semantically aligned strings therefore receive stronger routing pressure than distant or unrelated strings.

A bounded phase-alignment form can be written as:

$$g(|\phi_k - \phi_j|) = \frac{1 + \cos(\phi_k - \phi_j)}{2}$$

This keeps alignment in the interval $[0, 1]$, with fully aligned phases approaching 1 and opposed phases approaching 0. The purpose of this term is not to claim that AI behavior is literally angular, but to provide a compact routing geometry: related behaviors should be easier to blend, while unrelated behaviors should require stronger task pressure or governor justification.

V-channel routing becomes compute-aware when local cost is introduced. Each target string carries a local cost C_j , estimated from reasoning depth, output length, context use, retrieval pressure, tool likelihood, tension, stiffness, and other runtime signals. The governor discounts high-cost targets through an inverse-cost factor:

$$I_j = \frac{1}{1 + C_j}$$

The cost-aware propagation pressure is then:

$$P_{k \rightarrow j}^{cost} = P_{k \rightarrow j} \cdot I_j$$

This term makes expensive regions less attractive by default. A high-cost target string can still be activated when task value justifies it, but it no longer receives expansion simply because it is semantically related. The governor therefore prefers routes that are both behaviorally coherent and energetically affordable.

The routing process can be interpreted as a corridor-selection problem. The governor evaluates candidate transitions and favors paths with high task relevance, strong phase alignment, high coherence, and low avoidable cost. A useful route must satisfy several conditions at once:

- it must be relevant to the user's task;
- it must preserve or improve response coherence;
- it must remain within safety and policy constraints;
- it must respect the compute envelope;
- and it must not expand high-cost behavior unless that expansion is justified.

Reasoning depth is also governed along V-channels. If activation flows from source string k to target string j , the target's reasoning radius can update according to a bounded depth rule:

$$R'_j = \min (R_{max}, R_k \cdot h(P_{k \rightarrow j}^{cost}))$$

where R_k is the source reasoning depth, R_{max} is the maximum permitted radius under the compute envelope, and $h(\cdot)$ maps propagation strength into depth preservation or attenuation. When cost-aware propagation is strong, depth can be preserved along the corridor. When propagation is weak, misaligned, or expensive, depth is attenuated.

This prevents deep reasoning from spreading indiscriminately. A task that justifies technical depth may preserve radius through Depth, Precision, and Technical Detail, but the same depth should not automatically spill into unrelated strings such as Creativity or Initiative unless the task requires it. The channel therefore acts as a selective conduit: it carries depth where aligned and suppresses unnecessary expansion elsewhere.

The V-channel idea also provides a coherence diagnostic. Let Ω_{ij} denote the interior region of the membrane corresponding to the channel between strings i and j , and let $u(r, \theta, t)$ be the field activation inside the membrane. The average channel activation can be expressed as:

$$\bar{A}_{ij}(t) = \frac{1}{|\Omega_{ij}|} \int_{\Omega_{ij}} u(r, \theta, t) d\Omega$$

A simple channel-coherence ratio compares average activation inside the intended channel to the peak activation anywhere in the field:

$$Q_{ij}(t) = \frac{\bar{A}_{ij}(t)}{A_{peak}(t) + \epsilon}$$

High $Q_{ij}(t)$ means activation is concentrated inside a meaningful corridor. Low $Q_{ij}(t)$ means activation is scattered, spiking, or leaking into unrelated regions. This gives the governor a signal

beyond raw amplitude: it can ask not only whether the system is active, but whether the activation is organized.

This distinction is important for personalization. A user may prefer technically rigorous answers, but that does not mean every response should activate all analytical strings at maximum depth. V-channel routing lets the system learn and preserve preferred corridors while still allowing the governor to modulate cost, depth, and scope per task. Repeated coherent use of a corridor can make that route more available in future interactions, while noisy or low-value routes can decay.

In the condensed publication model, V-channel routing should therefore be understood as the paper’s mechanism for behavioral continuity. It gives the membrane a way to move through aligned modes smoothly, to preserve depth only where justified, and to suppress unrelated expansion without collapsing the whole response. The result is an adaptive AI identity that flows through structured, compute-aware reasoning corridors rather than switching between static personality presets.

5. Governor and Stability Control

The governor is the control layer that turns the radial identity membrane from a descriptive geometry into a bounded runtime system. Without a governor, behavioral strings could activate, propagate, and accumulate cost without a disciplined mechanism for deciding which expansions are useful, which are wasteful, and which are unsafe or unstable. In the proposed architecture, the governor sits above the behavioral membrane and evaluates each candidate state according to task value, local cost, phase alignment, coherence, policy validity, and stability pressure.

The governor does not modify the base model’s weights. It operates as an inference-time orchestration layer that translates membrane state into runtime controls such as reasoning depth, token budget, retrieval scope, tool-call threshold, response structure, tone, and explanation level. Its purpose is to let personalization become adaptive without becoming unbounded. The membrane may express user-shaped behavior, but the governor determines how far that behavior is allowed to expand, how much compute it may spend, and when it should return toward baseline.

At each update step, the governor reads the local state of each behavioral string:

$$s_i(t) = (A_i(t), \phi_i, R_i(t), \tau_i(t), \kappa_i(t))$$

where A_i is behavioral amplitude, ϕ_i is angular phase, R_i is reasoning depth, τ_i is dynamic tension, and κ_i is dynamic stiffness. These variables jointly determine how strongly a string is expressed, how far it extends into reasoning or context, how expensive it is to activate, and how resistant it is to further deformation.

A local cost function converts this state into an energetic footprint:

$$C_i = \lambda_R R_i^2 + \lambda_A A_i + \lambda_T T_i + \lambda_C \log(1 + \text{context}_i) + \lambda_\tau \tau_i + \lambda_\kappa \kappa_i$$

The squared depth term penalizes deep reasoning nonlinearly; the amplitude term captures expressive intensity; the tool term captures discrete agentic cost; the logarithmic context term captures diminishing returns from expanding context; and the tension and stiffness terms represent adaptive resistance. This cost is not merely recorded after a response is generated. It actively shapes routing, suppression, and future activation.

The governor's first routing principle is inverse-cost pressure. A candidate target string j receives an inverse-cost factor:

$$I_j = \frac{1}{1 + C_j}$$

or, in a sharper exponential form:

$$I_j = \exp(-\lambda_C C_j), \lambda_C > 0$$

Both forms make expensive regions less attractive. This does not mean high-cost reasoning is forbidden. It means that the system must justify expensive expansion through task value, coherence, safety, and expected user benefit. Cost becomes a routing force rather than a retrospective accounting metric.

Combining phase alignment, target activation, and inverse cost yields a robust propagation pressure:

$$P_{k \rightarrow j} = \text{sigmoid}(A_j) \cdot \frac{1 + \cos(\phi_k - \phi_j)}{2} \cdot I_j$$

This term gives the governor a compact selection rule: route toward strings that are already relevant, phase-aligned with the current corridor, and affordable under the current compute envelope. The sigmoid bounds amplitude attraction, the cosine term favors nearby or aligned behavioral phases, and the inverse-cost term discourages unnecessary expansion into expensive regions.

Reasoning depth is then updated through a bounded propagation rule:

$$R'_j = \min(R_{\max}, R_k \cdot h(P_{k \rightarrow j}))$$

where R_k is the source-string depth, $R_{\max}$ is the compute-envelope radius, and $h(\cdot)$ maps propagation pressure into depth preservation or attenuation. Strong, coherent, low-cost propagation can preserve depth across a V-channel. Weak, expensive, or misaligned propagation

attenuates depth. This prevents deep reasoning from spreading indiscriminately across unrelated behavioral modes.

After propagation, local cost is recomputed:

$$C'_j = \lambda_R(R'_j)^2 + \lambda_A A_j + \lambda_T T_j + \lambda_C \log(1 + \text{context}_j) + \lambda_\tau \tau_j + \lambda_\kappa \kappa_j$$

This closes the loop between behavior and cost. Every routed expansion has an immediate energetic consequence, and that consequence affects subsequent routing and suppression. In this sense, the governor is not only selecting a response style; it is managing a dynamical system whose state changes with every step.

The governor also applies global cost pressure. A conservative system might require total cost to be non-increasing during ordinary updates, but that would be too rigid for real tasks. Instead, the architecture uses a soft global cost behavior:

$$C'_{\text{total}} = C_{\text{total}} - \eta \Delta_{\text{efficiency}} + \gamma_{\text{task}} \Delta_{\text{task}}$$

where η weights efficiency pressure, $\Delta_{\text{efficiency}}$ estimates savings from routing or suppression, γ_{task} weights justified task expansion, and Δ_{task} is bounded task value. This allows the system to allocate additional compute when a task genuinely requires it while preserving a long-run bias toward efficiency.

The task-value term is deliberately bounded:

$$0 \leq \Delta_{\text{task}} \leq 1$$

This prevents task value from becoming an open-ended override. In practice, Δ_{task} should depend on explicit user intent, task criticality, expected utility, safety or compliance relevance, and whether additional reasoning is likely to improve the outcome. The governor may expand compute for a high-value technical review, legal or policy interpretation, or complex planning task; it should not repeatedly expand compute for low-value prompts that can be answered simply.

Suppression provides the recovery mechanism. Inactive, low-relevance, or high-cost regions contract according to global cost pressure:

$$R'_i = \sigma_R(C_{\text{total}}) R_i, A'_i = \sigma_A(C_{\text{total}}) A_i$$

with:

$$0 \leq \sigma_R(C_{\text{total}}), \sigma_A(C_{\text{total}}) \leq 1$$

As total cost rises, the suppression functions decrease, contracting optional radius and amplitude. This concentrates resources where the task requires them and pulls unrelated regions back

toward baseline. The governor should suppress optional verbosity, excessive branching, redundant context use, and low-value tool activity before suppressing essential safety, correctness, accessibility, or user-requested explanation.

The full update can be summarized as:

$$(A'_j, R'_j, C'_j) = F_{\text{gov}}(A_j, R_k, C_j, \tau_j, \kappa_j, \phi_k - \phi_j)$$

This unified update integrates behavioral attraction, phase alignment, inverse-cost pressure, bounded depth propagation, local cost recalculation, adaptive resistance, and suppression. The selected next state is not necessarily the most expressive state; it is the lowest-cost, policy-valid, coherent state that still satisfies the user's task.

Stability is tracked through a candidate Lyapunov-style energy function:

$$V = C_{\text{total}} + \alpha \sum_i (R_i - \bar{R})^2 + \beta \sum_i \tau_i + \delta \sum_i \kappa_i$$

where C_{total} is aggregate compute cost, R_i is each string's reasoning depth, $\bar{R}$ is average radius, τ_i is dynamic tension, κ_i is dynamic stiffness, and α, β, δ are positive weights. The first term discourages unnecessary compute expenditure. The second discourages unstable radial imbalance. The final terms discourage excessive tightness and rigidity in the membrane.

The desired ordinary-update condition is:

$$\Delta V = V' - V \leq 0 \text{ when } \Delta_{\text{task}} = 0$$

This does not mean the system can never expand. It means that expansion should be explicitly tied to task value. Under ordinary conditions, the governor should contract the membrane toward lower energy through inverse-cost routing, bounded depth, suppression, damping, and return-to-baseline pressure. Under justified task pressure, V may rise temporarily, but the system should subsequently dissipate that energy and return toward a stable operating region.

The stability interpretation has three zones. The center of the membrane is the low-energy recovery target. The interior field is the diagnostic space where coherent V-channel resonance can be distinguished from noisy activation. The outer compute envelope is the enforcement boundary that prevents unbounded reasoning depth, context expansion, tool use, or token growth. A stable governor keeps the system moving among these zones without allowing any one behavior to dominate permanently.

In short, the governor gives U.F.O. its central discipline: it allows adaptive personalization, but only through bounded, measurable, policy-valid transitions. It routes intelligence through coherent corridors, allocates compute when task value justifies it, suppresses optional expansion

when cost rises, and tracks energy so ordinary updates trend back toward stability. The result is a personalization system that is not merely expressive, but governable.

6. Deformable and Bidirectional Boundary Geometry

The radial identity membrane begins as a circular control surface, but a perfectly fixed circle is only a neutral abstraction. In realistic interaction, behavioral activation is uneven. Some strings sustain heavier task pressure than others; some regions require deeper reasoning, broader context, or stronger expressive amplitude; and some regions must contract under cost, suppression, or reduced coherence. For this reason, the publication version treats the membrane boundary as a deformable and bidirectional geometric surface rather than a fixed outer line.

In the earlier simplified model, the membrane boundary has constant radius:

$$r(\theta, t) = R_0$$

for all angular positions $\theta \in [0, 2\pi)$. This fixed boundary is useful for introducing the radial map, but it cannot represent asymmetric behavioral load. A user request that strongly activates Depth, Precision, and Technical Detail should not deform the membrane in the same way as a warm, concise interpersonal response. Likewise, a system under cost pressure should not merely “activate less”; it should be able to geometrically contract regions whose expansion is no longer justified.

The deformable boundary is defined as a polar curve:

$$r(\theta, t) = R_0 + \sum_{i=1}^n a_i(t) \phi_i(\theta)$$

where R_0 is the neutral radius, $a_i(t)$ is the sustained activation level of behavioral string s_i , and $\phi_i(\theta)$ is an angular basis function centered at the string’s home phase position θ_i . A simple Gaussian basis can be used:

$$\phi_i(\theta) = A_i \exp(-((\theta - \theta_i)^2 / 2\sigma_i^2))$$

where A_i is the maximum radial contribution of the string and σ_i controls the angular spread of influence. Narrow values of σ_i create localized protrusions, while broader values create smoother regional deformation across multiple related strings.

This formulation gives the architecture a direct mapping from behavioral activation to boundary shape. When a string is activated briefly, the membrane may ripple but not permanently deform. When a string remains active over a sustained interval, it stretches the local boundary outward. Adjacent activated strings superpose, forming ridges, protrusions, or broader lobes along the

membrane. The resulting boundary is therefore a geometric record of behavioral emphasis: it shows not only which strings are active, but how their activation reshapes the system's operating surface.

To prevent the membrane from accumulating permanent deformation after temporary tasks, each activation term decays when the corresponding string becomes inactive:

$$\dot{a}_i(t) = -\lambda_i a_i(t)$$

where λ_i is a string-specific relaxation rate. This decay rule lets the membrane adapt without becoming permanently distorted by every interaction. A high-value technical task may stretch the analytical region temporarily; after the task resolves, relaxation and governor suppression pull the membrane back toward baseline.

The boundary's local geometry provides several governor-readable signals. The first is radius deviation:

$$\Delta R(\theta, t) = R(\theta, t) - R_0$$

Positive ΔR indicates outward stretch. Negative ΔR indicates inward collapse. This bidirectionality is important. The membrane is not merely an activation bulge model in which more behavior always means more outward expansion. It is a governed control surface: cost pressure, suppression, tension gradients, or low coherence can pull the boundary inward.

The second signal is tangent direction or boundary slope:

$$\tau(\theta, t) = \frac{\partial R(\theta, t)}{\partial \theta}$$

Large values of $|\tau(\theta, t)|$ indicate rapid directional change across neighboring behavioral strings. These regions mark transition zones where the system is moving from one behavioral posture to another, such as from deep technical explanation toward concise summary, or from exploratory ideation toward structured recommendation. The tangent therefore encodes not only activation intensity, but the direction and sharpness of behavioral transition.

The third signal is local curvature:

$$\kappa(\theta, t) = \frac{\partial^2 R(\theta, t)}{\partial \theta^2}$$

High curvature corresponds to sharp geometric features such as points, ridges, folds, or concave regions. Curvature tells the governor how abruptly the membrane bends. A high-curvature outward ridge may indicate concentrated behavioral intensity, while a high-curvature inward fold

may indicate active suppression, instability recovery, or a region where cost pressure is collapsing unnecessary expansion.

These terms can be collected into a boundary-geometry signature:

$$G_{\text{geom}}(\theta, t) = (\Delta R(\theta, t), \tau(\theta, t), \kappa(\theta, t))$$

This tuple augments the existing string-level state variables with shape-aware information. Activation, cost, tension, and stiffness describe the internal dynamics of each behavioral string; boundary geometry describes how those internal pressures reshape the membrane as a whole. The governor can use this geometric signature to distinguish superficially similar activation states. Two units may have the same total activation energy, but one may be smoothly expanded while another forms a sharp ridge, unstable fold, or suppressed inward region. Their control implications are different.

The earlier tangent expressivity scalar can be retained as a normalized measure of boundary transition:

$$\mathcal{E}(\theta, t) = \left| \frac{r'(\theta, t)}{r(\theta, t)} \right|$$

High expressivity indicates a sharp relative change in the boundary; low expressivity indicates a smooth or uniform region. The governor can sample $\mathcal{E}(\theta, t)$ over k angular positions to form an expressivity profile $\mathbf{e}(t) \in \mathbb{R}^k$. This profile gives the system a compact way to detect where behavioral transitions are concentrated around the membrane.

A global asymmetry index summarizes how distorted the boundary has become:

$$\rho(t) = 1 - \frac{R_{\min}(t)}{R_{\max}(t)}$$

A perfectly circular membrane gives $\rho = 0$. A highly pointed or deeply asymmetric membrane pushes ρ upward. The governor can combine asymmetry, curvature, and expressivity into a geometric priority weight:

$$w_U(t) = w_0 + \alpha\rho(t) + \beta\kappa_{\max}(t) + \gamma \|\mathbf{e}(t)\|_2$$

where α , β , and γ determine the relative importance of asymmetry, maximum curvature, and expressivity. This weight does not automatically reward deformation; it tells the governor that the unit is geometrically active and may require closer routing, arbitration, or suppression depending on the sign and structure of the deformation.

Bidirectional deformation is the key extension. Outward stretch means the membrane is expressing sustained behavioral load. Inward collapse means the governor is actively reducing depth, cost, or instability in that region. The latter is not a failure condition. It is a stability signal. A region may collapse inward because optional explanation was suppressed, because a tool path was rejected, because context expansion became too expensive, or because the system detected incoherent activation and returned that sector toward baseline.

This gives the governor a richer interpretation layer:

- Outward stretch with low curvature suggests smooth task-aligned expansion.
- Outward stretch with high curvature suggests concentrated intensity that may require compute prioritization or monitoring.
- Inward collapse with low coherence suggests active suppression of unstable or low-value behavior.
- High tangent magnitude across adjacent sectors suggests a behavioral transition zone.
- Low asymmetry and low expressivity suggest relaxed, general-purpose operation.

The deformable boundary therefore turns the membrane from a visualization into a control surface. Boundary shape, slope, curvature, asymmetry, and inward collapse become measurable signals that help the governor route behavior, allocate compute, suppress unstable spikes, reinforce coherent V-channels, and return the system toward baseline after task pressure subsides.

The conceptual leap is that identity is no longer represented only by which strings are active or how strongly they activate. Identity is also represented by the shape the membrane assumes under load. A static circle describes a neutral operating envelope; a deformable and bidirectional boundary describes the live geometry of adaptive behavior. In this sense, the radial membrane becomes a governed geometric surface: outward stretch expresses sustained behavioral demand, inward collapse expresses stabilization, and the governor reads the boundary as part of the system's identity state.

7. Cost Taxonomy and Quality-Preserving Reduction

A central premise of the radial identity membrane is that cost should be treated as a governable runtime signal, not merely as an after-the-fact accounting measure. In many AI systems, cost is approximated by token count or latency. Those signals are useful, but incomplete. A personalized assistant may become expensive not only because it produces more words, but because it reasons more deeply, expands context, retrieves external information, invokes tools, generates alternatives, performs corrections, or recovers from unstable routing. For this reason, the governor models cost as a multidimensional telemetry surface.

The cost taxonomy separates runtime expense into distinct categories:

Cost Category	Description	Example Runtime Signal
Token/output cost	Cost of generated text or structured output	Output tokens, response length
Reasoning-depth cost	Cost of deeper inference, planning, or intermediate evaluation	Reasoning budget, step depth, internal planning passes
Context-window cost	Cost of admitting and processing prior context, memory, or documents	Context tokens, window size, document span
Retrieval cost	Cost of searching or loading external information	Retrieval calls, source count, ranking passes
Tool or agentic cost	Cost of invoking actions, tools, workflows, or external services	Tool calls, action plans, API invocations
Latency and system-load cost	Cost visible through time, throughput, or infrastructure pressure	Response latency, queue time, resource load
Correction or regeneration cost	Cost caused by low-quality first attempts or user correction loops	Regeneration count, edit rate, follow-up corrections
Instability-recovery cost	Cost of damping noisy activation, resolving route conflict, or returning to baseline	Suppression events, energy spikes, route resets

These categories are intentionally separable because different forms of cost have different product meanings. A long explanation, a large context window, a tool call, and a repeated correction loop may all increase system expense, but they should not be governed in the same way. A long explanation may be justified by user-requested transparency; a tool call may be justified by task completion; a repeated correction loop may indicate misrouting or poor first-pass quality. The governor therefore needs a cost representation that distinguishes useful expenditure from wasteful expansion.

For behavioral string i , the observable cost components can be collected into a normalized local cost vector:

$$\mathbf{c}_i(t) = [c_{\text{tok}}, c_{\text{depth}}, c_{\text{ctx}}, c_{\text{retr}}, c_{\text{tool}}, c_{\text{lat}}, c_{\text{corr}}, c_{\text{stab}}]$$

Each component is normalized to a comparable scale before aggregation. The governor can then project this vector into a scalar observable cost through explicit weights:

$$C_i^{\text{obs}}(t) = \mathbf{w}_C^\top \mathbf{c}_i(t)$$

where $\mathbf{w}_C$ defines the relative importance of each cost category for a task, product surface, tenant, or evaluation setting. For example, retrieval and tool calls may receive higher weights in enterprise workflows because they invoke external systems or access governed resources. Token and latency costs may receive moderate weights because they affect both infrastructure load and user experience. Correction and regeneration costs may receive special weight because they indicate that earlier compute may have been low quality, misrouted, or insufficiently governed.

This weighted projection connects implementation telemetry to the architectural cost variable C_i . The model does not require perfect observability of internal reasoning. It requires enough privacy-preserving telemetry to estimate whether a behavioral string or V-channel is becoming more expensive, more unstable, or less justified by task value. In simulation, the cost vector can be normalized synthetically. In prototype systems, it can be estimated from aggregate runtime signals such as tokens, latency, context size, retrieval frequency, tool calls, correction rate, and stability traces.

The most important distinction is between avoidable cost and justified cost. Avoidable cost is computing expansion that does not improve task success, safety, correctness, accessibility, user-requested depth, trust, or downstream usefulness. Examples include unnecessary verbosity, redundant context expansion, low-value retrieval, unsupported branching, avoidable tool calls, incoherent routing, and repeated regeneration caused by poor first-pass alignment. Justified cost is additional compute that improves the user’s outcome: deeper reasoning for a difficult problem, retrieval for factual grounding, tool use for task completion, explanation for transparency, or extra verification for safety and correctness.

This distinction prevents the architecture from becoming a cost-minimization heuristic. The governor should not minimize cost in isolation. It should minimize avoidable cost subject to quality constraints. A compact objective can be stated as:

$\min C_{\text{avoidable}}$ subject to task success, safety, correctness, accessibility, trust, and requested depth

In practical terms, the governor should suppress optional expansion first. It should reduce unnecessary length, excessive branching, redundant context, low-value retrieval, and speculative tool use before it suppresses essential safety reasoning, required accessibility support, critical context, or the level of explanation the user explicitly needs. A concise answer is only better when it remains complete enough for the task. A lower-cost route is only better when it preserves or improves outcome quality.

This cost taxonomy also improves validation. Instead of asking whether the system reduces total cost in the aggregate, researchers can ask which cost category changed and whether the change was beneficial. A reduction in output tokens may be valuable if task success remains stable, but harmful if correction rate rises. A retrieval increase may be justified if factual accuracy improves, but wasteful if it does not affect the answer. A latency increase may be acceptable for high-value technical reasoning but unacceptable for routine low-pressure tasks. The vector representation makes these tradeoffs measurable.

The cost layer therefore supports the broader purpose of the membrane. Behavioral strings are not only expressive modes; they are compute-bearing modes. V-channels are not only reasoning corridors; they are cost-shaped routes. The governor is not only a stabilizer; it is a quality-preserving allocator of runtime resources. By treating cost as multidimensional, weighted, and quality-constrained, the architecture can pursue compute efficiency without degrading the user experience.

In short, U.F.O. should be understood as a quality-preserving cost governor. It reduces wasteful expansion while allowing useful compute when the task warrants it. Extra compute is acceptable when it improves correctness, safety, coherence, accessibility, trust, or user value. It is suppressed when it reflects unnecessary length, redundant context, repeated correction, unstable routing, or low-value activity beyond the task’s needs.

8. Conclusion

This paper proposes a governed deformable radial membrane for compute-aware AI personalization. The model represents adaptive AI behavior as a geometric control surface in which phase-positioned behavioral strings jointly encode expression, reasoning depth, context use, tool likelihood, and compute cost.

The architecture links three mechanisms: a radial membrane that makes behavior both expressive and cost-bearing; V-channel routing with governor control for coherent, bounded activation; and a deformable bidirectional boundary whose shape, curvature, asymmetry, and inward or outward deformation become measurable control signals. The cost taxonomy further distinguishes justified compute from avoidable expansion, enabling efficiency without reducing task quality, safety, correctness, accessibility, or requested depth.

The work remains conceptual and mathematical, supported by Project Rainbow as an initial toy simulation rather than production validation. Future work should test the model through broader parameter sweeps, richer workload families, controlled prototypes, and comparisons against static personalization, response-length-only control, no-curvature routing, and no-governor baselines. If validated, the membrane could provide a control geometry for AI systems that are more personal, more efficient, and more governable without becoming less useful.

Appendix A. Mathematical Variable Summary and Model Properties

This appendix summarizes the principal mathematical variables used in the condensed publication model and states the intended properties of the architecture. It is provided as a compact reference for interpreting the equations in the main text; it is not intended as a proof of deployment validity.

Variable	Meaning	Role in the Model
$a_i(t)$	Activation coefficient of behavioral string i	Determines local behavioral expression and routing readiness
θ_i	Home phase position of string i	Defines angular locality and phase alignment between strings
$r_i(t)$	Reasoning radius or cognitive reach	Represents depth of inference, context use, retrieval, or planning
$c_i(t)$	Local compute cost of string i	Penalizes expensive expansion and informs inverse-cost routing
$\tau_i(t), \kappa_i(t)$	Dynamic tension and stiffness	Represent resistance, accumulated pressure, and deformation control
$R(\theta, t)$	Deformable boundary radius	Maps sustained behavioral load and suppression into boundary shape
$\Delta R(\theta, t)$	Radius deviation from neutral boundary	Indicates outward stretch or inward collapse
$K(\theta, t)$	Local boundary curvature	Identifies sharp ridges, folds, transition pressure, and suppressed regions
$E_{tan}(\theta, t)$	Tangent expressivity	Measures boundary transition intensity across adjacent sectors
$V(t)$	Lyapunov-style energy	Tracks stability, dissipation, and return-to-baseline behavior

Model Properties. The condensed architecture is intended to satisfy the following properties under simulation or prototype evaluation:

1. **Bounded activation:** behavioral coefficients remain within normalized limits and do not expand indefinitely.
2. **Phase-local coupling:** nearby or aligned strings receive stronger propagation than unrelated strings.
3. **Cost-aware routing:** high-cost strings are discounted unless task value, safety, correctness, or user-requested depth justifies activation.
4. **Quality-preserving reduction:** the objective is not cost minimization alone, but reduction of avoidable cost subject to task quality constraints.
5. **Bounded depth propagation:** reasoning radius may propagate through coherent V-channels but is capped by the compute envelope.

6. **Bidirectional deformation:** boundary regions may stretch outward under sustained behavioral load or collapse inward under suppression and recovery pressure.
7. **Stability bias:** ordinary updates should tend toward lower energy unless justified task value temporarily expands the membrane.
8. **Inspectability:** cost, curvature, asymmetry, expressivity, and energy traces provide observable signals for simulation, ablation, and prototype evaluation.

These properties define what the model should make measurable. They do not establish production performance by themselves. Rather, they specify what Project Rainbow and future prototypes should be able to test: whether governed routing reduces avoidable compute, whether useful depth is preserved, whether boundary geometry correlates with task pressure, and whether the system returns toward stable operation after expansion.

Appendix B. Integrated Interpretation and Validation Implications

B.1 Coupled Runtime Loop. The architecture should be interpreted as a closed runtime loop rather than as a static personalization layer. Behavioral strings shape the response posture; their activation produces local cost; the governor evaluates cost, coherence, policy validity, and task value; and the resulting suppression or expansion updates the membrane state. Personalization, compute allocation, and stability control therefore operate in the same state space.

B.2 Interior Flow and Boundary Shape. V-channels and the deformable boundary describe complementary parts of the same geometry. V-channels describe interior flow: how activation moves through related behavioral strings. The deformable boundary describes exterior shape: how sustained behavioral load, cost pressure, suppression, and recovery alter the operating envelope. Together, these mechanisms define both where behavior travels and how the system deforms under that behavior.

B.3 Interpreting Expansion and Collapse. Outward stretch and inward collapse are control signals, not value judgments. Outward stretch may indicate useful task-driven expansion, but it may also indicate costly overreach if unsupported by task value. Inward collapse may indicate suppression, stabilization, or recovery rather than failure. The governor interprets deformation by combining direction, curvature, coherence, asymmetry, expressivity, and cost.

B.4 Avoidable Cost and Justified Compute. The cost layer should be read as an allocation mechanism, not a mandate to minimize compute at all costs. High-cost behavior is justified when it improves correctness, safety, task completion, accessibility, trust, or user-requested depth. Low-cost behavior is insufficient when it causes missing context, weak reasoning, correction loops, or reduced usefulness. The objective is quality-preserving cost allocation.

B.5 Validation Implications. Project Rainbow and future prototypes should test whether V-channel routing improves behavioral coherence, whether inverse-cost pressure reduces avoidable compute, whether useful depth is preserved under cost control, whether boundary deformation

correlates with task pressure, and whether suppression restores stability without degrading quality. Ablations should compare the full architecture against static personalization, response-length-only control, no-curvature routing, and no-governor baselines.

Taken together, these interpretations frame the membrane as a governed systems architecture: behavior is routed, compute is allocated, boundary shape is monitored, and stability is recovered through measurable control signals. Appendix A defines the variables; Appendix B clarifies how those variables work together as an integrated runtime model.

Author Note

This condensed paper accompanies the full Dynamic Radial Membrane v2.6 research artifact. It presents the central architecture in a concise publication format while preserving the primary contribution: a governed, deformable radial membrane for compute-aware AI personalization. The full artifact contains extended mathematical development, simulation notes, appendices, version history, and exploratory extensions.

The work is submitted for Microsoft’s consideration because it addresses problems relevant to large-scale AI systems, including adaptive user experience, bounded compute, responsible personalization, runtime governance, telemetry, product integration, and trustworthy AI orchestration. It is not presented as a finished production system or empirical deployment result, but as a conceptual and mathematical research proposal for critique, refinement, further simulation, and potential prototype evaluation.

The purpose of this note is to clarify the paper’s intended use: to invite technical review of the model’s strengths, limitations, validation path, and possible relevance to future AI systems. The central thesis is that adaptive AI behavior may become more personal, efficient, and governable when represented as a bounded geometric control surface.

The author is grateful for the reviewers’ time, attention, and consideration in evaluating this research proposal.
